

PCA for Source Separation of Ventilation and Cardiovascular Activity in Electrical Impedance Tomography (EIT)

Introduction

In this exercise we use PCA for processing image sequences of thoracic electrical impedance tomography (EIT) signals. EIT is a non-invasive, radiation-free imaging modality which uses small alternating currents to measure bioimpedance of the thorax [1]. These measurements are then converted into image sequences of thoracic impedance changes representing ventilation (i.e., air exchange in the lungs) and cardiovascular activity (e.g., heart movement or blood volume changes in heart and lungs).

In order to analyze these data it is important to properly separate ventilation and cardiovascular activity. Besides common techniques such as frequency filtering or ECG-triggered averaging, PCA can be used for separating these two sources of signals. The present example uses the method proposed by Deibele et al. [2] for which the block diagram is shown below:

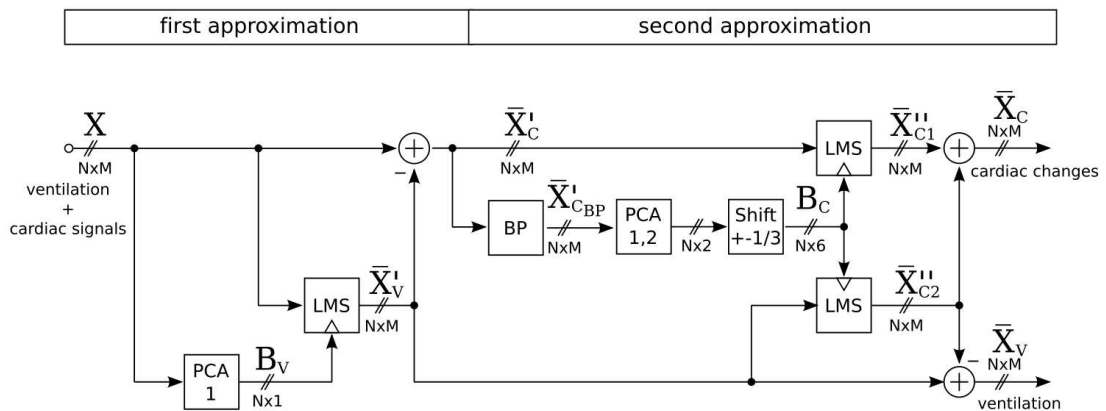


Figure 1. Flowchart of the method. PCA #: principal component number #, LMS: least-mean-squares fit, BP: bandpass, 'shift $\pm \frac{1}{3}$ ': input is phase shifted by $[-1/3, 0, 1/3]$ heart cycle to account for phase shifts introduced by the blood flow. N : number of processed frames, M : number of pixels in one frame.

References

- [1] I. Frerichs et al., "Chest electrical impedance tomography examination, data analysis, terminology, clinical use and recommendations: consensus statement of the TRanslational EIT developmeNt stuDy group," Thorax, vol. 72, no. 1, pp. 83–93, Jan. 2017, doi: [10.1136/thoraxjnl-2016-208357](https://doi.org/10.1136/thoraxjnl-2016-208357).
- [2] J. M. Deibele, H. Luepschen, and S. Leonhardt, "Dynamic separation of pulmonary and cardiac changes in electrical impedance tomography," Physiological Measurement, vol. 29, no. 6, pp. S1–S14, Jun. 2008, doi: [10.1088/0967-3334/29/6/S01](https://doi.org/10.1088/0967-3334/29/6/S01).

```
In [1]: import numpy as np
from scipy.io import loadmat
from scipy.signal import filtfilt, butter
from scipy.linalg import eig

import plotly.graph_objects as go
from plotly.offline import init_notebook_mode, iplot
from plotly.subplots import make_subplots
init_notebook_mode(connected=True) # initiate notebook for offline plot
```

```
from ecgdetectors import Detectors
```

```
In [2]: # Load data
data = loadmat('EIT_Data.mat')
t = data['tEit'].flatten()[data['IdxRange'].flatten().astype(bool)]
fs = 1/np.median(np.diff(t))
imgs_eit = data['Imgs']
b, a = butter(4, np.asarray([0.1, 12]), fs=fs, btype='bandpass')
imgs_eit = filtfilt(b, a, imgs_eit, axis=-1)
imgs_eit *= 1E3 # adapt scaling for plotting

# ECG data to be used for bonus question
ecg = {'time': data['Ecg'][0][0][2], 'value': data['Ecg'][0][0][1], 'fs': data['Ecg'][0][0][3]}
ecg_range = (ecg['time'] > t[0]) & (ecg['time'] < t[-1])
ecg['time'] = ecg['time'][ecg_range]
ecg['value'] = ecg['value'][ecg_range]

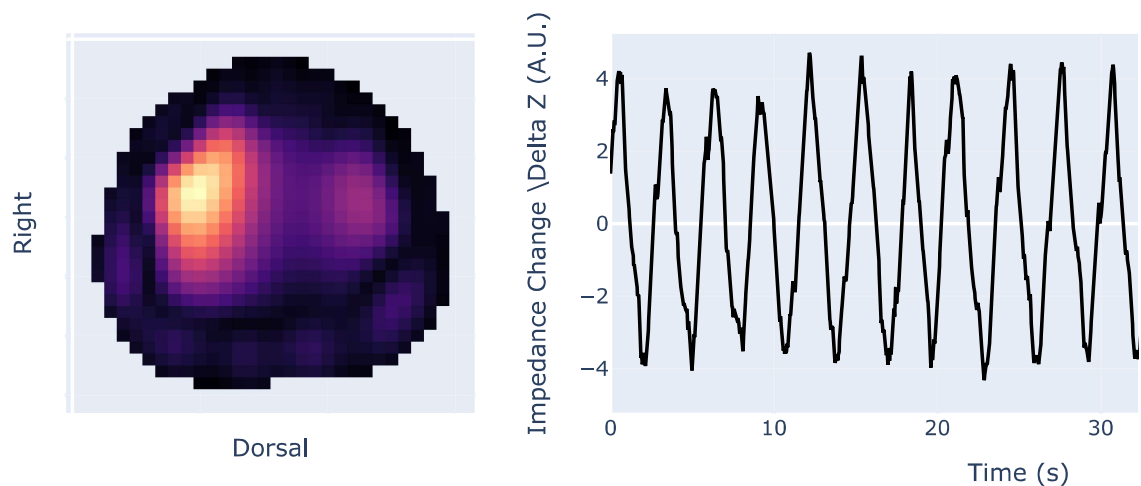
# force all timings to start at zero
t -= t[0]
ecg['time'] -= ecg['time'][0]
```

```
In [3]: # plot input data
fig = make_subplots(rows=1, cols=2, column_widths=[1, 2])
fig.update_layout(width=950, height=400)

fig.add_trace(go.Heatmap(z=np.std(imgs_eit, axis=-1), zmin=0,
                           showscale=False, colorscale='magma'), row=1, col=1)
fig.update_yaxes(title='Right', showticklabels=False, autorange="reversed", row=1, col=1)
fig.update_xaxes(title='Dorsal', showticklabels=False, row=1, col=1)
fig.update_layout(title='Overall EIT Activity')

fig.add_trace(go.Scatter(x=t, y=np.nansum(imgs_eit, axis=(0, 1)),
                          name='Overall Sum Signal', line_color='black'), row=1, col=2)
fig.update_xaxes(title='Time (s)', row=1, col=2)
fig.update_yaxes(title='Impedance Change \Delta Z (A.U.)', row=1, col=2)
```

Overall EIT Activity



```
In [4]: def compute_principal_components(X):
# perform PCA and compute principal components (pc) and eigenvalues of X
A = np.dot(X.transpose(), X) # covariance matrix
[eigenvalues, eigenvectors] = eigh(A)
pc = np.dot(X, eigenvectors)
return pc, eigenvalues
```

```

def estimate_cardiac_frequency(s, fs):
    sign_cardiac = np.sign(s)
    # find where zero crossings occurred
    index = np.where(np.diff(sign_cardiac) == 2)[0]
    # interpolate to a sub-sample resolution
    pos = index + s[index + 1] / (s[index + 1] - s[index])
    # interpolate RR interval values
    rr = np.diff(pos) / fs
    return 1/np.median(rr)

def lms(A, B):
    if B.ndim == 1:
        B = np.asmatrix(B).transpose()
    tmp = np.linalg.solve(np.dot(B.transpose(), B), B.transpose())
    return np.dot(np.dot(B, tmp), A)

# separate ventilation and cardiovascular activity using PCA
# according to the algorithm by Deibele et al., PhysMeas, 2008
# https://dx.doi.org/10.1088/0967-3334/29/6/S01
imgs_tmp = np.reshape(imgs_eit, [-1, imgs_eit.shape[-1]])
are_valid_pixels = np.all(~np.isnan(imgs_tmp), 1)
X = imgs_tmp[are_valid_pixels, :].transpose()

# first approximation (see block diagram)
X = X - np.repeat(np.reshape(np.mean(X, 0), [1, -1]), X.shape[0], 0)
PC1, lambda1 = compute_principal_components(X)
Bv = PC1[:, -1]
Xv_ = lms(X, Bv)
Xc_ = X - Xv_

# second approximation (see block diagram)
Xc_ = Xc_ - np.repeat(np.reshape(np.mean(Xc_, 0), [1, -1]), Xc_.shape[0], 0)
b, a = butter(6, np.asarray([0.92, 4.6]), fs=fs, btype='bandpass')
Xc_bp = filtfilt(b, a, Xc_, axis=0)
PC2, lambda2 = compute_principal_components(Xc_bp)
Bc_ = PC2[:, -2:]
fc = estimate_cardiac_frequency(Bc_[ :, 0], fs)

# create cardiac template functions
Bc = np.hstack((Bc_, np.roll(Bc_, int(fs/fc/3), 0),
                    np.roll(Bc_, -int(fs/fc/3), 0)))
Xc1 = lms(Xc_, Bc)
Xc2 = lms(Xv_, Bc)
Xc = (Xc1 + Xc2).transpose()
Xv = (Xv_ - Xc2).transpose()

# cardiovascular activity
imgs_card = np.full(imgs_tmp.shape, np.nan)
imgs_card[are_valid_pixels, :] = Xc
imgs_card = imgs_card.reshape(imgs_eit.shape)
# ventilation activity
imgs_vent = np.full(imgs_tmp.shape, np.nan)
imgs_vent[are_valid_pixels, :] = Xv
imgs_vent = imgs_vent.reshape(imgs_eit.shape)

```

```

In [5]: # plot ventilation activity
fig = make_subplots(rows=1, cols=2, column_widths=[1, 2])
fig.update_layout(width=950, height=400)

fig.add_trace(go.Heatmap(z=np.std(imgs_vent, axis=-1), zmin=0,
                        showscale=False, colorscale='magma'), row=1, col=1)
fig.update_yaxes(title='Right', showticklabels=False, autorange="reversed", row=1, col=1)
fig.update_xaxes(title='Dorsal', showticklabels=False, row=1, col=1)
fig.update_layout(title='Ventilation Activity')

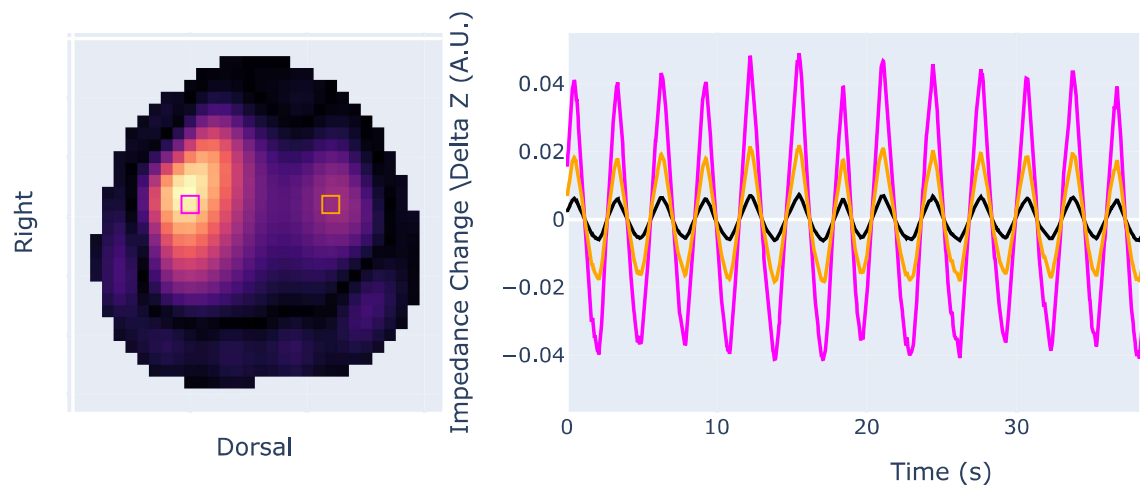
fig.add_trace(go.Scatter(x=t, y=np.nanmean(imgs_vent, axis=(0, 1)),
                        name='Mean Signal', line_color='black'), row=1, col=2)
fig.update_xaxes(title='Time (s)', row=1, col=2)
fig.update_yaxes(title='Impedance Change \Delta Z (A.U.)', row=1, col=2)

```

```
# add example signals in ??? regions
regions = {'Region RL': ([10, 14], 'magenta'), 'Region LL': ([22, 14], 'orange')}
for reg, tmp in regions.items():
    fig.add_scatter(x=tmp[0][0], y=tmp[0][1], mode='markers', marker_symbol='square-open',
                   marker_size=10, legendgroup=reg, marker_color=tmp[1], name=reg, row=1, col=1)
    fig.add_trace(go.Scatter(x=t, y=imgs_vent[tmp[0][1], tmp[0][0], :], legendgroup=reg,
                             showlegend=False, name=reg, line_color=tmp[1]), row=1, col=2)

fig.show()
```

Ventilation Activity



```
In [6]: # plot cardiovascular activity
fig = make_subplots(rows=1, cols=2, column_widths=[1, 2])
fig.update_layout(width=950, height=400)

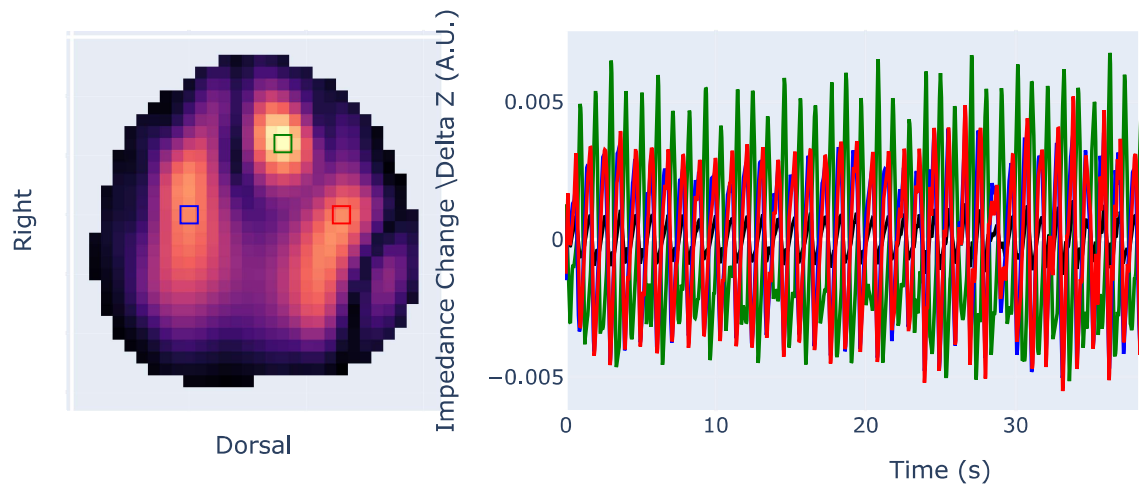
fig.add_trace(go.Heatmap(z=np.std(imgs_card, axis=-1), zmin=0,
                          showscale=False, colorscale='magma'), row=1, col=1)
fig.update_yaxes(title='Right', showticklabels=False, autorange="reversed", row=1, col=1)
fig.update_xaxes(title='Dorsal', showticklabels=False, row=1, col=1)
fig.update_layout(title='Cardiovascular Activity')

fig.add_trace(go.Scatter(x=t, y=np.nanmean(imgs_card, axis=(0, 1)),
                          name='Mean Signal', line_color='black'), row=1, col=2)
fig.update_xaxes(title='Time (s)', row=1, col=2)
fig.update_yaxes(title='Impedance Change \Delta Z (A.U.)', row=1, col=2)

# add three example signals in ??? regions
regions = {'Region A': ([18, 9], 'green'), 'Region B': ([10, 15], 'blue'), 'Region C': ([23, 15])
for reg, tmp in regions.items():
    fig.add_scatter(x=tmp[0][0], y=tmp[0][1], mode='markers', marker_symbol='square-open',
                   marker_size=10, legendgroup=reg, marker_color=tmp[1], name=reg, row=1, col=1)
    fig.add_trace(go.Scatter(x=t, y=imgs_card[tmp[0][1], tmp[0][0], :], legendgroup=reg,
                             showlegend=False, name=reg, line_color=tmp[1]), row=1, col=2)

fig.show()
```

Cardiovascular Activity



Exercise Questions

Please provide your answers directly below each question.

Question 1

Determine the frequency of the ventilation activity (i.e., the respiratory rate), both expressed in **Hz** and **respirations/min**.

```
In [7]: vent_signal = np.nanmean(imgs_vent, axis=(0, 1))
vent_centered = vent_signal - np.nanmean(vent_signal)
vent_window = np.hanning(vent_centered.size)
vent_fft = np.fft.rfft(vent_centered * vent_window)
vent_freqs = np.fft.rfftfreq(vent_centered.size, d=1/fs)
vent_mag = np.abs(vent_fft)
f_vent = vent_freqs[np.argmax(vent_mag)]
print(f"Ventilation rate: {f_vent:.3f} Hz = {f_vent*60:.1f} respirations/min")
```

Ventilation rate: 0.320 Hz = 19.2 respirations/min

Question 2

Determine the frequency of the cardiovascular activity, both expressed in **Hz** and **beats/min**.

```
In [8]: card_signal = np.nanmean(imgs_card, axis=(0, 1))
card_centered = card_signal - np.nanmean(card_signal)
card_window = np.hanning(card_centered.size)
card_fft = np.fft.rfft(card_centered * card_window)
card_freqs = np.fft.rfftfreq(card_centered.size, d=1/fs)
card_mag = np.abs(card_fft)
f_card = card_freqs[np.argmax(card_mag)]
print(f"Cardiac rate: {f_card:.3f} Hz = {f_card*60:.1f} beats/min")
```

Cardiac rate: 0.959 Hz = 57.5 beats/min

Question 3

Determine the following three values:

- i) the maximal amplitude of ventilation activity;
- ii) the maximal amplitude of cardiovascular activity; and
- iii) the ratio between i) and ii), i.e., ventilation vs cardiovascular activity.

```
In [9]: vent_amp = float(np.nanmax(np.abs(vent_signal)))
card_amp = float(np.nanmax(np.abs(card_signal)))
ratio = vent_amp / card_amp
print(f"Max ventilation amplitude: {vent_amp:.4e} ΔZ (A.U.)")
print(f"Max cardiac amplitude: {card_amp:.4e} ΔZ (A.U.)")
print(f"Amplitude ratio (vent/card): {ratio:.2f}")
```

```
Max ventilation amplitude: 7.6811e-03 ΔZ (A.U.)
Max cardiac amplitude: 1.4271e-03 ΔZ (A.U.)
Amplitude ratio (vent/card): 5.38
```

Question 4

Determine the eigenvalues of the first three principal components resulting from the first PCA (see variable `PC1`).

```
In [10]: top3 = lambda1[-1:-4:-1]
for idx, val in enumerate(top3, start=1):
    print(f"Eigenvalue PC{idx}: {val:.6f}")
```

```
Eigenvalue PC1: 107.586887
Eigenvalue PC2: 2.111489
Eigenvalue PC3: 0.803152
```

Question 5

Similar to Question 4, determine the two most dominant frequencies for the first three principal components. *Note that this would lead to a total of 6 values, 2 frequencies for each of the 3 PCA components. However, for some components only one dominant frequency might be present.*

For all 3 PCA components, which one is rather related to ventilation or cardiovascular activity?

```
In [11]: pc_results = {}
pc_spectra = {}
for k in range(3):
    comp = PC1[:, -(k + 1)] # take kth component (sorted by descending eigenvalue)
    comp_centered = comp - np.mean(comp) # remove DC offset
    comp_window = np.hanning(comp_centered.size)
    comp_fft = np.fft.rfft(comp_centered * comp_window)
    comp_freqs = np.fft.rfftfreq(comp_centered.size, d=1/fs)
    comp_mag = np.abs(comp_fft)
    idx_sorted = np.argsort(comp_mag)[-2:][::-1] # pick indices of two largest peaks
    pc_results[f"PC{k+1}"] = [(comp_freqs[i], comp_mag[i]) for i in idx_sorted] # save peak info
    pc_spectra[f"PC{k+1}"] = (comp_freqs, comp_mag) # keep entire spectrum for plotting

for pc_name, freqs in pc_results.items():
    print(pc_name) # show component label
    for freq, mag in freqs:
        label = "ventilation" if abs(freq - f_vent) < 0.05 else "cardiovascular" # classify peak
        print(f" {freq:.3f} Hz (|A|={mag:.2f}) → {label}")
print(f"If we only consider the visualization of the spectrogram we can also take into consideration")

fig = make_subplots(rows=3, cols=1, shared_xaxes=False, subplot_titles=list(pc_spectra.keys()))
for row, (pc_name, (freqs, mags)) in enumerate(pc_spectra.items(), start=1):
    mask = freqs <= 5 # ne garder que les fréquences <= 5 Hz
    fig.add_trace(go.Scatter(x=freqs[mask], y=mags[mask], name=pc_name, showlegend=False),
                  row=row, col=1)
    fig.update_yaxes(title_text="|FFT|", row=row, col=1)
    fig.update_xaxes(title_text="Frequency (Hz)", range=[0, 5], row=row, col=1) #usefull signal
```

```
fig.update_layout(height=800, width=850, title="Spectra of first three principal components")
fig.show()
```

PC1

0.320 Hz ($|A|=154.61$) → ventilation

0.340 Hz ($|A|=131.68$) → ventilation

PC2

0.959 Hz ($|A|=14.49$) → cardiovascular

0.979 Hz ($|A|=11.30$) → cardiovascular

PC3

0.320 Hz ($|A|=7.54$) → ventilation

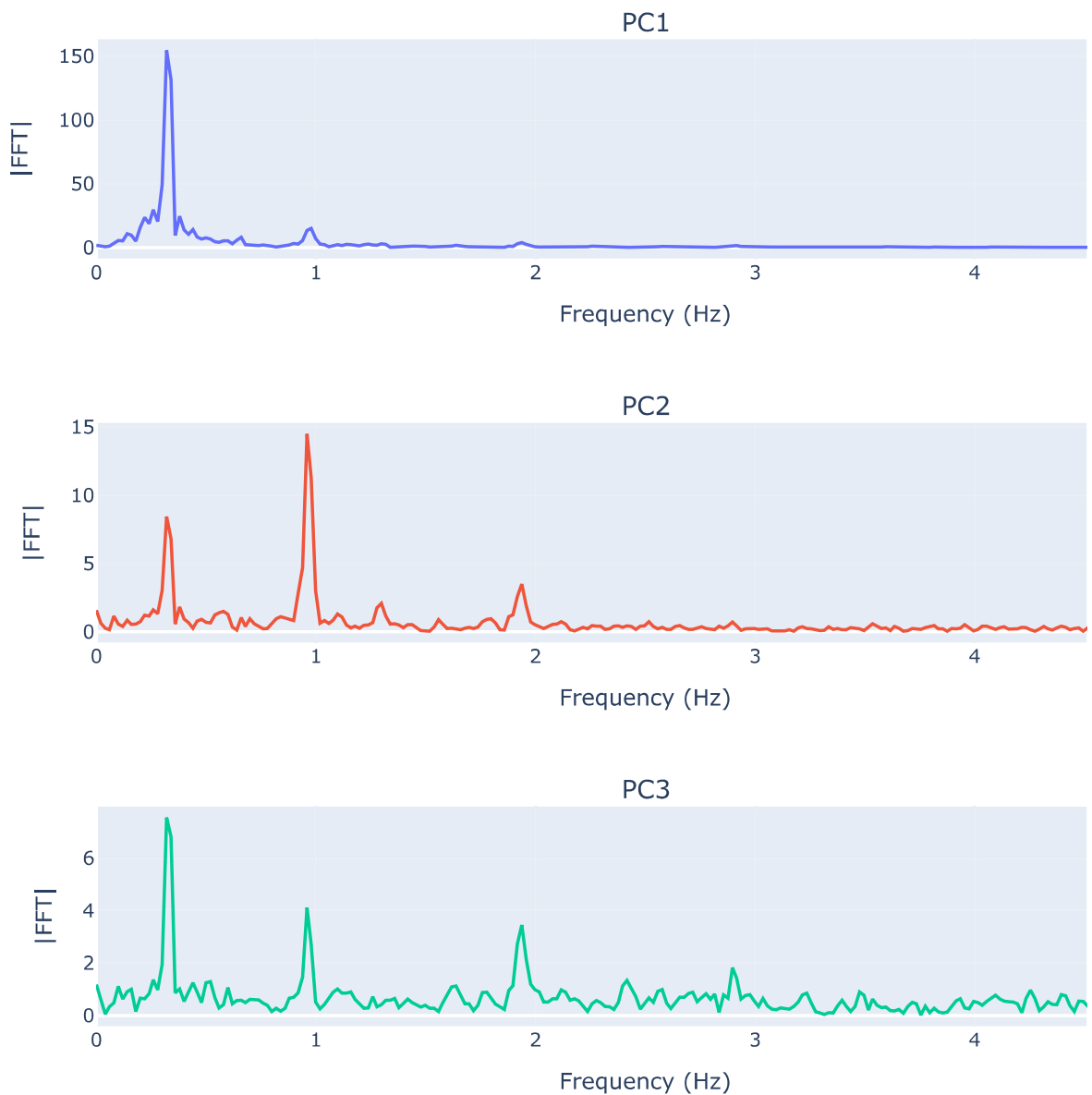
0.340 Hz ($|A|=6.81$) → ventilation

If we only consider the visualtion of the spectrogram we can also take into consideration for PCA2 the pic at 0.32Hz wich correspond to ventilation.

Same for PCA3, we have a pic at 0.96Hz that correspond to cardiovascular.

The fact is that we in reality have two peaks close to each other

Spectra of first three principal components



The three example signals (Regions A to C: **green, blue, red**) of cardiovascular activity show the impedance change over time. Can you guess the underlying anatomical structure for each of the three example signals (Regions A to C: **green, blue, red**).

```
In [12]: print("In the plots already done above. The green region matches the heart because it shows the
```

In the plots already done above. The green region matches the heart because it shows the biggest pulsatile signal.

The blue and red regions show lower but in phase pulses, which fits to the two lungs.

Question 7 - *Bonus Question* (not graded)

Can you detect the QRS peaks (e.g., using `qrs = Detectors(fs).engzee_detector(ecg)`) on the ECG signal (see variable `ecg`) and plot them together with the cardiovascular EIT activity?

```
In [13]: detectors = Detectors(ecg['fs'])
qrs_samples = detectors.engzee_detector(ecg['value'])
qrs_times = ecg['time'][qrs_samples]
qrs_on_eit = np.interp(qrs_times, t, card_signal)

fig = go.Figure()
fig.add_trace(go.Scatter(x=t, y=card_signal, name="Mean cardiovascular"))
fig.add_trace(go.Scatter(x=qrs_times, y=qrs_on_eit, mode="markers",
                        marker=dict(color="crimson", size=8),
                        name="QRS detections (ECG)"))
fig.update_layout(title="Cardiovascular vs. ECG QRS detections",
                  xaxis_title="Time (s)",
                  yaxis_title="Impedance change  $\Delta Z$  (A.U.)")
fig.show()
```

Cardiovascular vs. ECG QRS detections

